Lowering resource usage with foot and systemd

Rohit Goswami · 2024-06-02 · workflow · tools

> *Melding `systemd` and `foot` for rapid configuration updates and reduced memory consumption in `sway` with persistent `tmux` state*

Background

Since I often work in strong sunlight, I often want to reconfigure my terminal to use a light or a dark scheme depending on the time of day. In the process, I decided to fiddle around with custom `systemd` targets, since `sway` isn't really good at managing long running processes like my terminal (`foot`) server.

Starting point

Some years ago I ended up with a rather ad-hoc scratchpad configuration, reproduced for clarity (current config here):

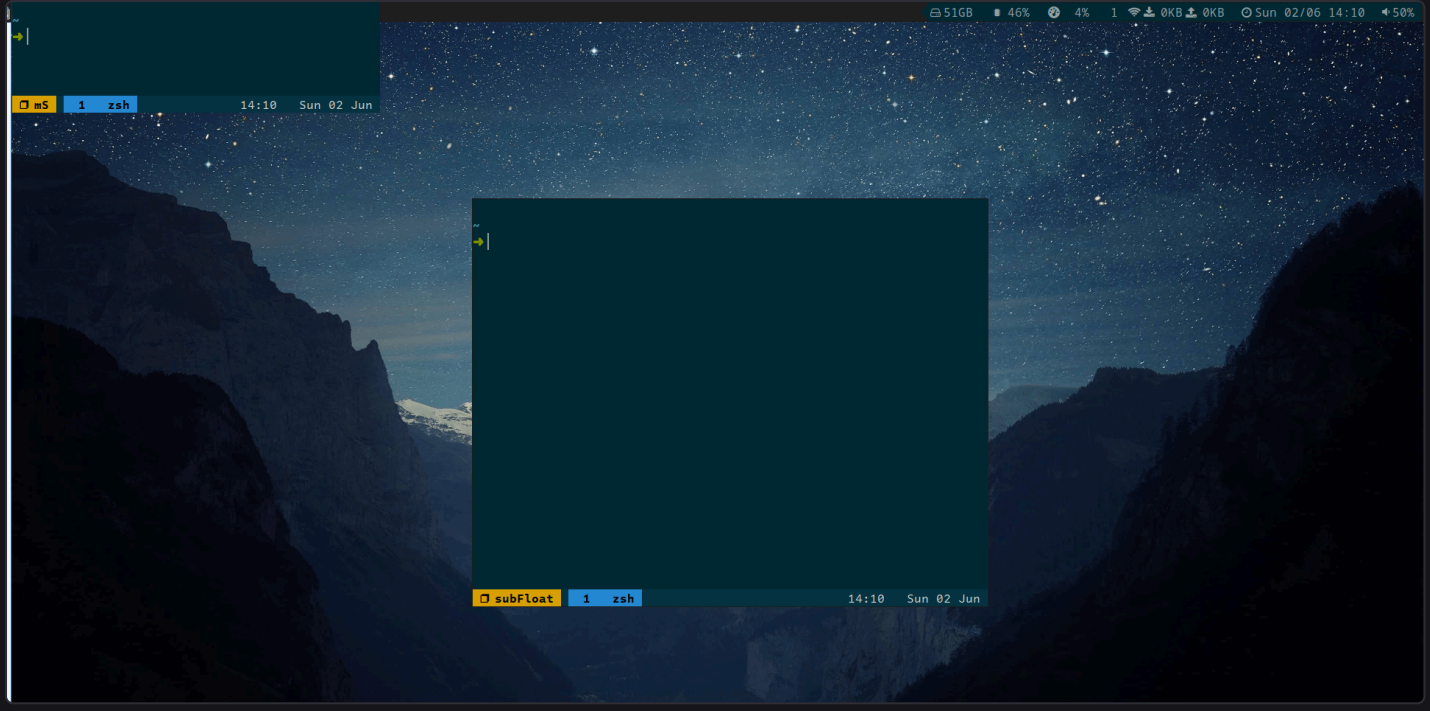
```
# The main scratchpad in the upper left
for_window [app_id="mS"] {
    move scratchpad
    move position 0 0
    resize set 50 ppt 35 ppt
    floating enable
    border pixel 1
}
bindsym F1 [app_id="mS"] scratchpad show; move position 0 0, resize set 50 ppt 35 ppt
exec_always foot --app-id="mS" -e tmux new -A -s mS

# The secondary scratchpad in the center
for_window [app_id="subFloat"] {
    move scratchpad
    move position center
    floating enable
    border pixel 1
}
bindsym F5 exec swaymsg [app_id="subFloat"] scratchpad show || exec foot --app-id="subFloat" -e tmux new -A -s
subFloat
```

Which essentially boils down to:

- Put a scratchpad in the upper left bound to `F1` and resize it to a long thin window
  - Attach or start a `tmux` session here called `mS`
- Put another scratchpad in the center, bound to `F5`
  - Attach or start a `tmux` session called `subFloat`

When everything works (e.g. at start-up) it essentially leads to a scratchpad setup like so:



Additionally, I also have some additional setp for managing terminals:

```
set $term_server foot -s
set $term footclient
exec_always $term_server
bindsym $mod+Return exec $term
```

Which ideally starts a `foot` server, and subsequently spawns `footclient` instances on demand.

Annoyances

This setup served me quite well, for several years I had no major issues whatsover, though there were minor annoyances, some of which eventually broke the camel's back.

Memory consumption

The problem is perhaps immediately evident in the narrowed down specification:

```
exec_always foot --app-id="mS" -e tmux new -A -s mS
# ...
exec_always foot --app-id="subFloat" -e tmux new -A -s subFloat
```

Right of the bat, it is rather evident that this configuration will spawn two instances of `foot`, one for each scratchpad.

Figure 1: Snapshot of memory consumption (from `btop`), clients take around 10x less memory

On its own, given that I mostly use relatively high-end machines, this was not really a problem, more an aesthetic annoyance.

Applying `foot` configurations

If the main (`sway`) server is restarted to apply a configuration update it will not propagate to the scratchpad unless those are also restarted, and `sway` isn't meant to handle long running processes. Essentially, applying a change of theme via `foot.ini` (crrent config here):

```
# include = ~/.config/foot/foot.d/theme-acario-light.ini
include = ~/.config/foot/foot.d/theme-solarized-dark-normal-brights.ini
```

Would entail a whole `killall -9 foot` song and dance which was also rather annoying, especially under the current configuration, the `tmux` sessions would also die with the `foot` instances.

`exec_always` doesn't restart failed processes

Nor should it, as the documentation explicitly mentions, it is basically like `exec` but it also gets re-executed after a reload. For something like a terminal server service, this isn't exactly what is required at all. Afterall:

- It is independent of the `sway` configuration

Or it really should be. On the other hand, some of the settings for the scratchpads were baked into the configuration. Having to reload the `sway` configuration (and redo the `tmux` state) just because aesthetic changes needed to be propagated is asinine, and finally annoyed me enough to make the switch to a real service manager.

Using `systemd` to offload management

The solution is to abstract the configuration for the main and helper scratchpads out of the configuration and also offload management of the process lifetime to `systemd`. User configurations for `systemd` live in `$HOME/.config/systemd/user` and require some standard commands for interacting with them:

```
# Reload user units, every time files are changed
systemctl --user daemon-reload
# Start and enable a service
systemctl --user --now foot-server.service
# Reload or restart, also when files are changed
systemctl --user reload-or-restart foot-server.service
# Check the status
systemctl --user status foot-server.service
```

There are a few common stanzas which are needed for a minimal service file:

```
[Unit]
Description=
[Service]
ExecStart=
Restart=on-failure
[Install]
WantedBy=default.target
```

With much more extensive documentation found in the `man` pages (or online, here).

Terminal Server

The ordering of the services are also fairly obvious, both the scratchpads (and indeed, the sway terminals) need one primary `foot-server` instance to be running at all times. The easiest approach is to reuse `XDG_RUNTIME_DIR` which is typically the same as `/run/user/$(id -u $USER)/`, so the server runs via:

```
/usr/bin/foot --server=/run/user/%U/foot-server.sock
```

Since `foot -s` doesn't fork itself, the default `Type=simple` will suffice.

Additionally, we will need to ensure that all the `footclient` instances we call use the same server socket (as shown in Sway Scratchpads and Config).

Scratchpad clients

The clients themselves are equally simple, except that the `[UNIT]` stanza will specify they require `foot-server.service` and will be run after `foot-server.service`.

```
/usr/bin/footclient --server-socket=/run/user/%U/foot-server.sock --app-id="mS" -e tmux new -A -s mS
```

The full configurations are reproduced in the next section.

`tmux` sessions

Additionally, since the goal is to reload `foot` often via simple `killall -9 foot` to apply new configurations, it makes sense to add a `tmux` server into the mix, which will also be part of the requires and after keys for the scratchpad clients.

For this we need to consider some additional options:

```
[Service]
Type= # simple (default), forking, oneshot
ExecStop=
RemainAfterExit=yes
```

Some notes on service types:

- The server will be a `forking` service, since `tmux start-server` essentially puts itself in the background.
- The clients should be `oneshot`, since they should complete before other dependent services start[^fn:1]
- We need to set remain after exit since we have a "rollback" for our service (`ExecStop`) and since we modify system state, i.e. we don't want to re-run the service actions without stopping[^fn:2]

False starts

I thought I wanted to use sockets (online man-page), but for a constantly running set of terminals there is very little point. On my machine, the `foot` server takes around 20MB-31MB while each client instance is below 1.5MB, so having a a socket for either the server or the clients made little to no sense.

Complete configurations[^fn:3]

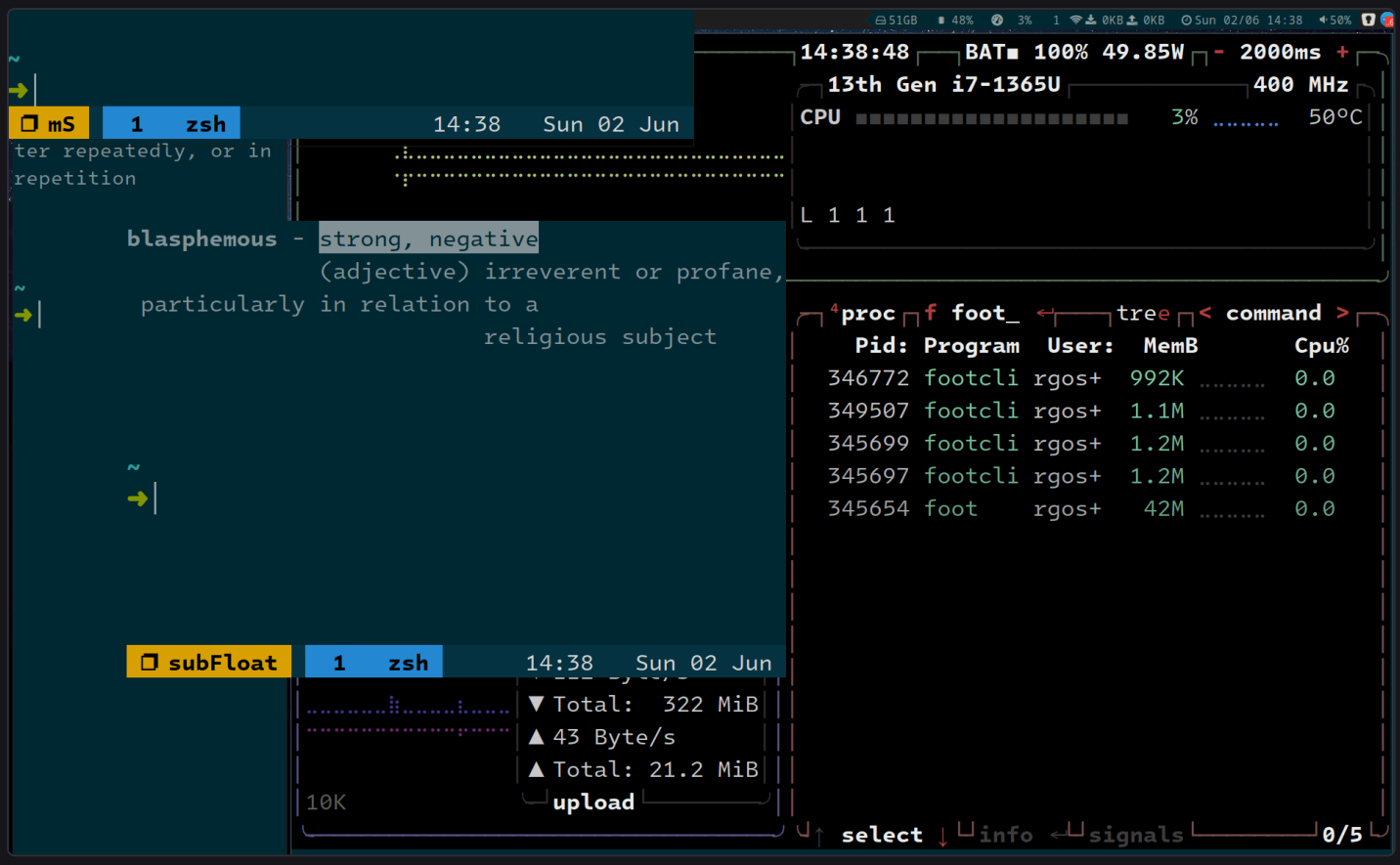


Figure 2: Final single server many client view (via `btop`)

Sway Scratchpads and Config

The final `sway` scratchpad setup is essentially:

```
# Settings
set {
 $ms_pos border none, move position 0 0, resize set 50 ppt 35 ppt
 $sf_pos border pixel 1, move position center
 $start_ms systemctl --user start foot-ms.service
 $start_sf systemctl --user start foot-subfloat.service
}
# The main scratchpad in the upper left
for_window [app_id="mS"] {
    move to scratchpad
    floating enable
    $ms_pos
}
bindsym F1 exec swaymsg [app_id="mS"] scratchpad show || $start_ms, $ms_pos

# The secondary scratchpad in the center
for_window [app_id="subFloat"] {
    move to scratchpad
    floating enable
    border pixel 1
}
bindsym F5 exec swaymsg [app_id="subFloat"] scratchpad show || $start_sf, $sf_pos
```

With the bindings changed to point to the same socket as the process:

```
# Remove exec_always $term_server, infact get rid of it all together, since systemd handles it now
set $term footclient --server-socket=$XDG_RUNTIME_DIR/foot-server.sock
```

Systemd services

`foot` setup

```
[Unit]
Description=Foot terminal server

[Service]
ExecStart=/usr/bin/foot --server=/run/user/%U/foot-server.sock
Restart=on-failure

[Install]
WantedBy=default.target
```

```
[Unit]
Description=Foot client for tmux session mS
After=foot-server.service tmux-session-ms.service
Requires=foot-server.service tmux-session-ms.service

[Service]
ExecStart=/usr/bin/footclient --server-socket=/run/user/%U/foot-server.sock --app-id="mS" -e tmux new -A -s mS
Restart=on-failure

[Install]
WantedBy=default.target
```

```
[Unit]
Description=Foot client for tmux session subFloat
After=foot-server.service tmux-session-subfloat.service
Requires=foot-server.service tmux-session-subfloat.service

[Service]
ExecStart=/usr/bin/footclient --server-socket=/run/user/%U/foot-server.sock --app-id="subFloat" -e tmux new -A
-s subFloat
Restart=on-failure

[Install]
WantedBy=default.target
```

`tmux` setup

```
[Unit]
Description=Tmux server
After=network.target

[Service]
Type=forking
ExecStart=/usr/bin/tmux start-server
ExecStop=/usr/bin/tmux kill-server
RemainAfterExit=yes

[Install]
WantedBy=default.target
```

```
[Unit]
Description=Tmux session mS
After=tmux-server.service
Requires=tmux-server.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/tmux new-session -s mS -d
ExecStop=/usr/bin/tmux kill-session -t mS

[Install]
WantedBy=default.target
```

```
[Unit]
Description=Tmux session subFloat
After=tmux-server.service
Requires=tmux-server.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/tmux new-session -s subFloat -d
ExecStop=/usr/bin/tmux kill-session -t subFloat

[Install]
WantedBy=default.target
```

Conclusions

Perhaps on modern machines, with many gigabytes of RAM, there is no real benefit to this approach, however, given that I use a lot of terminals and don't always reach for `tmux`, for me this works out very nicely. It also helps handling configuration updates to `foot`, since all instances use the same server.

Moreover, this setup greatly facilitates changing themes, a feature that is particularly useful for those who work in varying lighting conditions, like moving between indoor and outdoor environments. For people who consistently work in the same environment, this might not be a significant advantage. However, the persistent `tmux` services is likely to be useful in any case.

Nevertheless, the minor annoyances have been quelled, and I had never really worked with `systemd` services before barring a few backup setups, so this was pretty gratifying.

[^fn:1]: There is an excellent visual explanation of the differences b/w simple and oneshot types here") by Thomas Stringer [^fn:2]: Aside from the official documentation, this SO answer is quite nice [^fn:3]: for now, updates will probably appear only on my Dotfiles repo